

THIS IS THE ONLY LEVEL GAME REPORT

Name: Alperen Ulu

YouTube Link: <https://youtu.be/HFjE55S1BJ8>

Game Description: This Is the Only Level (2009)

In this game, the goal is to reach the exit pipe and complete five creatively designed stages. Although the level layout stays visually the same, each stage brings a unique gameplay mechanic, such as constant jumping, inverted controls, or modified gravity. The game takes place in a world affected by gravity, and the player controls a blue elephant using the arrow keys to move and avoid obstacles. At the start of each level, the elephant drops from a pipe, and to open the door blocking the exit pipe, a button must be found and pressed. Each stage has a different twist: **Stage 1** is normal; **Stage 2** inverts the controls; **Stage 3** features constant jumping with higher gravity and speed; **Stage 4** requires pressing the button five times; and in **Stage 5**, everything except the start pipe, exit pipe, and player is invisible. The game is completed once all five stages are successfully passed.

Gameplay Elements:

The gameplay includes several core elements that define the player's experience. **Obstacles** are solid surfaces that the player can stand on but cannot pass through, shaping the level's layout and requiring careful navigation. **Spikes** are deadly hazards that instantly kill the player upon contact, causing the stage to restart from the beginning. The game features two types of pipes: the player falls from the **start pipe** at the beginning of each stage, and the goal is to reach the **exit pipe**, which can only be accessed after unlocking the **door** in front of it. This door acts as a barrier and must be opened by pressing a **button** located somewhere in the stage. Once the button is activated, the door unlocks, allowing the player to complete the level. Additionally, there is a **settings area** on the screen that includes a **timer** to track how long the game has been played, a **death counter** to show how many times the player has died, and a **stage tracker** to display the current stage. It also offers options to **reset** or **restart** the game, along with a button to receive a **clue** or **help** for the current stage.

Implementation Details

In this assignment, **Object-Oriented Programming (OOP)** concepts are applied in an interactive and practical way. The game is designed using five main classes, each responsible for a different feature of the game, such as controlling the player, handling the game logic, managing stages, and drawing visuals. All **data fields** are declared as **private** to **ensure encapsulation and data security**. To access or modify these fields, when necessary, **public getter and setter methods** are provided. Each class also includes **private helper methods** that handle specific tasks and make up the core logic of that class.

Classes:

Main

This class creates five **Stage** objects, each with specific characteristics such as gravity, movement speed, control keys, text messages. These stages are then added to an **ArrayList<Stage>** called **stages**, which is later used by the **Game** class to manage stage progression. After setting up the stages, a **Game** object is initialized with this list, and the **play()** method is called to start the game.

Stage

Represents a game stage with specific properties such as gravity, velocity, control keys, and visual elements. Each stage can have unique behavior, colors, and helpful hints for the player. This class stores information for one stage of the game, including **physical values (gravity, velocity)**, **player controls (key codes)**, and **helps/clues** to assist the player.

Table 1: Stage UML Diagram

Stage	
- gravity: double - velocityX: double - velocityY: double - stageNumber: int - rightCode: int - leftCode: int - upCode: int - clue: String - help: String - color: Color - <u>colorArray: Color[]</u>	- Vertical acceleration affecting the player. - Player's speed along the x-axis. - Player's speed along the y-axis. - ID for the stage. - Right key code for movement. - Left key code for movement. - Up key code for movement. - Short stage hint. - Helpful tip for solving the stage. - Visual color of the stage. - Static array storing 5 random stage colors.
+ Stage(gravity: double, velocityX: double, velocityY: double, stageNumber: int, rightCode: int, leftCode: int, upCode: int, clue: String, help: String) + getStageNumber(): int + getGravity(): double + getVelocityX(): double + getVelocityY(): double + getKeyCodes(): int[] + getClue(): String + getHelp(): String + getColor(): Color + <u>setColorArray(): void</u>	- Initializes all stage parameters and prepares the color array. - Returns stage ID. - Returns gravity value. - Return movement speed in x axis. - Return movement speed in y axis. - Returns movement key codes. [rightCode, leftCode, upCode] - Return clue text. - Return help text. - Returns the stage color. - Randomizes and sets colors for all stages.

Explanations of Game Mechanics by The Methods in Stage Class

+ **getClue()**: When the player does not click the help box, a brief clue about the stage is shown (e.g., "A bit bouncy here").

+ **getHelp()**: When the player clicks the help box, a more detailed hint about the stage is displayed (e.g., "You jump constantly").

+ **getColor()**: Retrieves the stage color from the **colorArray**. If the stage index exceeds the array bounds, it returns black as an error color. However, this should never occur in normal gameplay, as it is handled in the main game while loop.

+ **setColorArray()**: Uses the **Random** class to generate random red, green, and blue values for each color. In the last stage, the selected color cannot be used due to the background being white in that specific stage.

Player

Represents a player character in the game, with position, velocity, and directional properties. This class handles the player's movement, collision detection, and drawing of the elephant character. The player's position is controlled via x and y coordinates, and the elephant's movement direction is updated based on input and the current stage of the game.

Table 2: Player UML Diagram

Player	
- x: double - y: double - width: double - height: double - velocityY: double - elephantDirection: String	- The x-coordinate of the player's center. - The y-coordinate of the player's center. - The width of the player character. - The height of the player character. - The vertical velocity - The image file name representing the current direction the elephant is facing.
+ Player(x: double, y: double) + setX(x: double): void + setY(y: double): void + getX(): double + getY(): double + setVelocityY(velocityY: double): void + getVelocityY(): double + respawn(): void + setElephant(direction: char, stage: Stage): void + getHalfWidth(): double + getHalfHeight(): double + draw(): void	- Initializes the player's position and sets default size and direction. - Sets the x-position of the player's center. - Sets the y-position of the player's center. - Returns the current x-position. - Returns the current y-position. - Updates the vertical velocity of the player. - Returns the current vertical velocity. - Resets the player to a default position and velocity - Sets the direction/image of the elephant based on movement and game stage. - Returns half of the player's width - Returns half of the player's height. - Renders the elephant on screen at the current position with the current direction.

Explanations of Game Mechanics by The Methods in Player Class

+ **setVelocityY()** / **getVelocityY()**: In the game, there are two types of velocityY. The first, in the Stage class, represents the constant upward velocity when the player jumps. The second, in the Player class, represents the instantaneous vertical speed affected by gravity, and is managed using these methods.

+ **respawn()**: The respawn method for the player resets the player's center coordinates to the starting point, located at coordinates (130, 465) — the midpoint of the start pipe. It sets the vertical velocity to zero and sets the elephant's facing direction to its default value, "ElephantRight.png" (elephant looking right).

+ **setElephant()**: The setElephant method turns the elephant to the right when the right key is pressed and to the left when the left key is pressed, except during the second stage. In this stage, the controls are inverted: the elephant turns left when the right key is pressed, and right when the left key is pressed.

Map

The Map class handles the game environment for a given stage. It controls player movement, collisions, obstacles, buttons, spikes, pipes, and door mechanics.

Table 3: Map UML Diagram

Map	
- Stage stage - Player player - int[][] obstacles - int[] button - int[] buttonFloor - int[][] startPipe - int[][] exitPipe - int[][] spikes - int[] door - int buttonPressNum - boolean isDoorOpen - boolean isInAir - boolean isButtonVisible - boolean isStageChanged - int deaths	- The current level's layout and properties. - The player object controlled in the game. - Coordinates of all impassable objects. - Position of the pressable button. - Position of the floor the button is on. - Coordinates for pipe entry points. - Coordinates for pipe exit points. - Coordinates of spike traps. - Position of the stage door. - Counter for how many times button was pressed. - Flag to track if door is open. - Displays whether the player is currently in the air. - Visibility state of the button. - Displays whether the stage transition is triggered. - Tracks the number of player deaths in a single stage.
+ Map(Stage stage, Player player) + void movePlayer(char direction) + boolean changeStage() + void pressButton() + void restartStage() + Stage getStage() + Player getPlayer() + boolean getStageChange() + int getDeathNumber() + void draw() - boolean checkCollision(double nextX, double nextY, int[] obstacle) - boolean isComingFromCorner(int[] obstacle) - int findCollisionNumber(double nextX, double nextY) - void openDoor() - double[] nextPosition() - void correctLocation(double nextX, double nextY, int[] obstacle) - boolean isInAir()	- Initializes the map with a stage and player. - Moves player and checks collisions. - Returns whether the stage has changed. - Triggers button logic and possibly opens door. - Resets the stage to the beginning. - Returns current stage. - Returns player instance. - Gets flag for stage change (isStageChanged). - Returns the number of deaths. - Renders all map elements on screen. - Detects if player collides with a specific obstacle. - - Checks for edge/corner interactions. - - Identifies which obstacle was collided with. - - Sets the flag to open the door. - Calculates the player's next coordinates based on input. - Adjusts player location to prevent passing through objects. - - Determines if the player is airborne (not standing on anything).

Explanations of Game Mechanics by The Methods in Map Class

+ **changeStage()**: When the player's next position overlaps with the inner area of the exit pipe (**exitPipe[1]**), the stage change is triggered. The method sets the **isStageChanged** variable (which tracks stage transition) to true.

+ **pressButton()**: When the player's next position touches the button, the **buttonPressNum** variable (which counts how many times the button has been pressed) is incremented by one.

+ **restartStage()**: Respawns the player using the respawn method from the **Player** class. Resets the door to its original coordinates and sets **isDoorOpen** to false. It also resets the button and increments the deaths counter by one.

– **nextPosition()**: Calculates the player's next position using values such as velocity, gravity, and current position, retrieved from the **Player** and **Stage** classes. When the player presses the right or left key, the position is updated by **velocityX** in that direction.

```
if (StdDraw.isKeyPressed(stage.getKeyCodes()[0])){
    nextX += velocityX;
}
if (StdDraw.isKeyPressed(stage.getKeyCodes()[1])){
    nextX -= velocityX;
}
if ((StdDraw.isKeyPressed(stage.getKeyCodes()[2]) || getStage().getStageNumber() == 2) && !isInAir){
    velocityY += (stage.getVelocityY() + gravity);
    nextY += velocityY;
}
if (isInAir){
    velocityY += gravity;
    nextY += velocityY;
}
```

Figure 1: Next Position Calculation

If the up key is pressed, or if it is **Stage 3** and the player touches the ground, the **y-position** is increased by the vertical velocity. If the player is in the air, gravity is applied and **velocityY** is reduced by the gravity value. (See Figure 1.) These calculations do not modify the actual values in the **Stage** or **Player** classes but are returned to be used within the **Map** class.

– **checkCollision()**: Checks whether the player's next position plus its width overlaps with the current obstacle's boundaries. If it does, the method returns true; otherwise, false. It takes the player's next position and the current obstacle as parameters.

– **isComingFromCorner()**: Determines whether the player is colliding with an obstacle from the corner by comparing the player's coordinates with the obstacle's edges. (See Figure 2.) Takes the current obstacle as a parameter and returns a **Boolean**.

```
if (player.getY() < obstacle[1] && player.getX() < obstacle[0]){
    return true;
}
```

Figure 2: Corner Collision Check

– **findCollisionNumber()**: Uses the **checkCollision** method to count how many obstacles the player is colliding with. If a collision is detected, a local **collisionNumber** variable is incremented. Takes the next position as a parameter and returns the number of collisions.

– **openDoor()**: When the button is pressed, the door opens gradually from top to bottom. The door's height decreases by two units in each frame until its top and bottom coordinates become equal, indicating the door is fully opened.

– **correctLocation()**: Takes the player's next position and the currently processed obstacle. If the player is on one side of the obstacle and is predicted to intersect with it on the next move, the player's

```
if (player.getY() < obstacle[1] && nextY > player.getY()){
    player.setY(obstacle[1] - player.getHalfHeight());
    player.setVelocityY(0);
}
```

Figure 3: Location Correction in Y-axis

location is corrected. For **x-axis collisions**, the player is moved to the left or right edge of the obstacle, depending on their direction of movement. For **y-axis collisions**, the logic is similar, but in addition, the player's vertical velocity (**velocityY**) is set to zero—since this axis is affected by gravity, not just by key presses. (See Figure 3)

– **isInAir()**: After the player's position is corrected, the method checks whether the player is directly on top of an obstacle. If so, it returns true, indicating the player is airborne. Otherwise, it returns false. (See Figure 4)

```
for (int[] obstacle : obstacles) {
    if (player.getY() - player.getHalfHeight() == obstacle[3]
        && player.getX() + player.getHalfWidth() > obstacle[0]
        && player.getX() - player.getHalfWidth() < obstacle[2]) {
        return false;
    }
}
return true;
```

Figure 4: Airborne Player Detection

+**movePlayer()**: This is the core method of the class; all other methods serve to support its functionality. It takes the movement direction as a parameter. First, it calculates the player's next position and sets the direction the elephant is facing. It then checks whether the button

```
for (int[] obstacle : obstacles){
    if (isComingFromCorner(obstacle) && findCollisionNumber(nextX,nextY) > 1){
        continue;
    }
    if ((checkCollision(nextX, nextY, obstacle))) {
        correctLocation(nextX,nextY,obstacle);
    }
}
```

Figure 5: Obstacle Collision Check

has been pressed. If the button has been pressed the required number of times (typically once, but five times in **Stage 4**), the door opens. If the door is still closed and the player collides with it, the player's position is corrected to prevent passing through. If the player collides with the button, the button is considered pressed and becomes invisible. Next, the method iterates through all obstacles using a for loop and checks for collisions using the **checkCollision()** method. Before correcting the player's position, the method checks whether the detected collision point is visually a **"real" corner**. Since the game obstacles are rectangles stacked together, players may perceive them as continuous surfaces rather than distinct corners. To prevent visual inconsistency, if two collisions are detected, one at a side and one at a corner, the side collision is prioritized to avoid reacting to a **"fake" corner**. (See Figure 5) After handling obstacles, the method checks for spike collisions. If a spike is touched, the player is respawned, **buttonPressNum** is reset to 0, the door is closed, and the elephant's direction is set to default (right). The death counter is also incremented. If the stage has changed (**checked via changeStage()**), the player is respawned again, the door is reset, and **isStageChanged** is set to true. Finally, movement is executed. First, the method determines whether the player is in the air. Then, four conditional blocks handle motion based on the input direction. When the direction is right or left it goes in that direction by the amount of **velocityX** of stage class. If up key is pressed or it is the **Stage 3**, and the player touches the ground, its position is updated upwards by the amount of **velocityY** of player stage. Before that, **velocityY** of player class is set bearing in mind the gravitational effect. If player is in air and player is neither going to right nor left, gravity is incremented by given value in **Stage** class. Neglecting right and left direction because this method is called by **handleInput()** method in the game folder with the directions ('R', 'L', 'U', 'N'). If they are not neglected, the corresponding if block activates three times by **"right, left and nowhere"** directions. So, gravity is calculated three times brings about an error. (e.g. Figure 6)

```
if (direction == 'R') {
    player.setX(player.getX() + stage.getVelocityX());
}
if (direction == 'L') {
    player.setX(player.getX() - stage.getVelocityX());
}
if ((direction == 'U' || getStage().getStageNumber() == 2) && !isInAir) {
    player.setVelocityY(stage.getVelocityY() + stage.getGravity());
    player.setY(player.getY() + stage.getVelocityY());
}
if (isInAir && (direction != 'R' && direction != 'L')){ // N stands for nowhere.
    player.setVelocityY(player.getVelocityY() + stage.getGravity());
    player.setY(player.getY() + player.getVelocityY());
}
```

Figure 6: Player Movement

+draw(): Renders all map elements, excluding the settings interface at the bottom. The default font is **Sans Serif** with a size (punto) of **16**. For the stage-passing banner that displays the message “You passed the stage, but is the level over?”, the font size is increased to **32**. Spikes are drawn with orientation based on the wall they are attached to, using the following angles (See Figure 7):

- **Ceiling:** 0°
- **Left Wall:** 90°
- **Floor:** 180°
- **Right Wall:** 270°

In the final stage, all elements (except the player and pipes) are rendered in white.

```
if (i == 0 && stage.getStageNumber() != 4) {
    angle = 90;
    StdDraw.picture(centerX, centerY, filename: "Spikes.png", height, width, angle);
}
else if ((i == 1 || i == 2 || i == 3) && stage.getStageNumber() != 4) {
    angle = 180;
    StdDraw.picture(centerX, centerY, filename: "Spikes.png", width, height, angle);
}
else if (i == 4 && stage.getStageNumber() != 4) {
    angle = 270;
    StdDraw.picture(centerX, centerY, filename: "Spikes.png", height, width, angle);
}
else {
    if(stage.getStageNumber() !=4) {
        angle = 0;
        StdDraw.picture(centerX, centerY, filename: "Spikes.png", width, height, angle);
    }
}
```

Figure 7: Spike Placement

Game

The Game class manages the overall gameplay loop including input handling, stage transitions, timing, death tracking, and showing the game state.

Table 4: Game UML Diagram

Game	
• - int stageIndex • - ArrayList<Stage> stages • - int deathNumber • - int gameTime • - int resetTime • - boolean resetGame • - boolean isGameOver • - boolean isHelpActivated • - boolean isMousePressedOnce • - int startTime • - int currentTime • - int pausedTime • - int finishTime • - int previousDeaths • - int frameTime • - int pauseDelay	- Index of the current stage. - List of all stages in the game. - Tracks total player deaths. - Total time spent in the game. - Time marker for when game was reset. - Flag to trigger full game reset. - Flag to indicate if the game is completed. - Whether help text is currently shown. - Prevents double-detection of clicks. - Game start timestamp. - Timestamp of the current frame. - Total paused duration between levels. - Timestamp when the game was completed. - Total deaths carried over between levels. - Delay between frames (25ms). - Delay after each stage (2000ms).
• + Game(ArrayList<Stage> stages) • + void play() • • + int getStageIndex(Map map) • + Stage getCurrentStage() • - void draw(Map map) • - void handleInput(Map map) • - boolean isMouseClicked() • - int[] getTime(int timeInMs)	- Initializes game with an array list of stages. - - Main game loop; controls logic, drawing, and interaction. - Gets the current stage index. - Retrieves the active stage. - Draws the current screen depending on game state. - Processes user input for movement and UI. - Checks if the mouse was clicked once. - Converts milliseconds into minutes, seconds,

Explanations of Game Mechanics by The Methods in Game Class

-isMouseClicked(): This method detects a single mouse click event, ensuring that a mouse button press is only registered once per click, even if the button is held down across multiple frames. Local variable “current” stores the current mouse press status using **StdDraw.isMousePressed()**. A click is only registered if the current press is true, and the mouse wasn’t already pressed in the previous frame (**!isMousePressedOnce**). After checking, **isMousePressedOnce** is updated to reflect the current mouse status, ensuring that the next frame knows whether the mouse was already pressed. (See Figure 8)

-getTime(): This method is a utility function that converts a time value from milliseconds into a human-readable format consisting of **minutes, seconds, and centiseconds**. It calculates the number of full minutes by dividing the total milliseconds by 60,000, then determines the remaining seconds and tenths of a second through modular arithmetic. The result is returned as an integer array with three elements: **{minutes, seconds, milliseconds/10}**. (See Figure 9) This formatted time is used in the game to display how long a player has been playing. It helps present time clearly in **MM:SS:MS** format.

-handleInput(): This method takes a Map object as its parameter. It begins by setting the local variables **mouseX** and **mouseY**, which represent the coordinates of the mouse press. After retrieving the current stage, the method determines the player's movement direction by checking key presses and calling **map.movePlayer()** accordingly. The player can move in four directions, represented by characters: **'R' (right), 'L' (left), 'U' (up), and 'N' (no movement)**. If one of the right, left or up key codes is pressed, player moves in that direction. If no movement key is pressed, the player is moved using the 'N' direction to account for gravity and in-air behavior. (See Figure 10) Next, the method handles mouse interactions with the game's UI. If the cursor is within the **Help button's area (centered at coordinates 250, 85)** and a mouse click is detected using the **isMouseClicked()** method, the **isHelpActivated** flag is set to true. As a result, the help text for the current stage is displayed instead of the clue. If the Restart button (located at 550, 85) is clicked, the current stage is restarted via **map.restartStage()**. Additionally, the **startTime** is recalculated so that the timer reflects only active playtime, and **pausedTime** is reset to zero. Clicking the **Reset button (centered at 400, 20)** sets the **resetGame** flag to true, indicating a

```
private boolean isMouseClicked() { 3 usages
    boolean current = StdDraw.isMousePressed();
    boolean clicked = current && !isMousePressedOnce;
    isMousePressedOnce = current;
    return clicked;
}
```

Figure 8: Mouse Click Detection

```
private int[] getTime(int timeInMs){ 6 usages
    int minute;
    int second;
    int millisecond;
    minute = timeInMs / (60 * 1000);
    second = (timeInMs % (60 * 1000)) / 1000;
    millisecond = (timeInMs % 1000)/10;
    return new int[]{minute,second,millisecond};
}
```

Figure 9/Equation 1: Game Time Calculation

```
if (StdDraw.isKeyPressed(stage.getKeyCodes()[0])){
    map.movePlayer( direction: 'R');
}
if (StdDraw.isKeyPressed(stage.getKeyCodes()[1])){
    map.movePlayer( direction: 'L');
}
if (StdDraw.isKeyPressed(stage.getKeyCodes()[2])) {
    map.movePlayer( direction: 'U');
}
else {
    map.movePlayer( direction: 'N');
}
```

Figure 10: Key Press Handle/Direction Assignment

full game reset. This reinitializes major variables such as the time counters and death count and returns the player to Stage 1. Finally, once all keyboard and mouse inputs are processed, the current stage index is updated using **getStageIndex(map)**, ensuring the game's logic stays parallel with the map's active stage.

+play(): This method contains the main game loop, managing the overall flow of the game. It handles player movement, stage transitions, time tracking, game states, and screen rendering. The loop runs continuously using **while(true)**, allowing real-time response to player input and in-game events. At the beginning, the game screen is set up by defining the canvas size and scale. Double buffering is enabled with

StdDraw.enableDoubleBuffering() to reduce flickering during updates. A Player object is created at the starting position, and a Map is initialized with the first stage and the player. The start time is recorded, and stage colors are assigned using

Stage.setColorArray(). Inside the loop, the system time is regularly updated. The total number of deaths is calculated by adding deaths from the current stage (via the Map) and previous stages. If the game is over, an end screen is shown with a congratulatory message (Figure 11). The player can restart the game by

pressing 'A', which resets variables like stage index, death count, and time. Pressing 'Q' exits the game using **System.exit(0)** (Figure 12). If the game is still active, several actions are performed in sequence: The game loop waits for the duration of **frameTime** to

control update speed. Then, the screen is cleared, and both keyboard and mouse inputs are handled through **handleInput()**. Finally, the current map is drawn, followed by the game UI using the **draw(map)** method. When **map.getStageChange()** is true, **stageIndex** increments to move to the next stage. It logs the stage time and checks if all stages are complete. If so, the game ends and saves the total time. Otherwise, it pauses for 2 seconds, resets the player and map, and updates variables like **isHelpActivated** and **previousDeaths**. If **resetGame** is true, the game fully resets it restarts from Stage 1, resets timers and death count, respawns the player, and applies a 2-second transition. The reset flag is then cleared. After each loop, **StdDraw.show()** refreshes the display.

-draw(): This method renders one of three states (**Game Over, Reset, Play State**) using the **Map** object. If **resetGame** is true, a green "RESETTING THE GAME..." banner (font 50) appears. If **isGameOver**, it shows a "Game Over" screen (font 32) with deaths and total time (font 20). During gameplay, it draws buttons (**Help, Restart, Reset**), current stage info, deaths, and time. It also displays either help or clue text based on whether help mode is active.

How the Map and Game Classes Interact with Each Other and Other Classes



Figure 11: Congratulatory Message

```
if (StdDraw.isKeyPressed(KeyEvent.VK_A)){
    isGameOver = false;
    stageIndex = 0;
    player = new Player( x: 130, y: 465);
    map = new Map(stages.get(0), player);
    startTime = (int) System.currentTimeMillis();
    currentTime = (int) System.currentTimeMillis();
    pausedTime = 0;
    resetTime = 0;
    finishTime = 0;
    deathNumber = map.getDeathNumber();
    previousDeaths = 0;
}
if (StdDraw.isKeyPressed(KeyEvent.VK_Q)) {
    System.exit( status: 0);
}
```

Figure 12: Game Over Controls

In this project, the Game and Map classes are two of the most important parts of the system. They work together to run the platformer game smoothly. Each of them has a different job, but they need to communicate with each other and with other classes like Player and Stage to function properly.

Interaction Between Game and Map: The Game class manages the overall game flow, including starting, updating, and transitioning between stages. It initializes a **Map** with the current **Stage** and Player, then repeatedly calls methods like **map.draw()**, **map.movePlayer()**, and **map.getStageChange()** to handle gameplay. When a stage is completed, it loads the next one with a new Map and Player.

Interaction Between Map and Stage: The Map class contains a **Stage** object. This stage holds information about the level, like gravity, controls, or which obstacles exist. **Map** uses this data to calculate movement and collisions. For example, when the player jumps or hits a spike, Map uses **Stage** values to decide what should happen next.

Interaction Between Map and Player: The **Map** class also keeps track of the **Player**. It handles everything about the player's movement, position, and collisions. It decides whether the player can move, whether they hit something, or whether they die. The **Map** calls methods from the **Player** class to move the character, apply gravity, and reset the player's position when needed.

Designed Stage in Detail (Do You Remember?)

In this level, the player must reach the exit pipe by remembering the layout from earlier stages. All game elements like platforms, spikes, buttons, and doors are white. Only the player and the start and exit pipes are colored. The stage name, **"Do You Remember?"**, is the main clue. The layout is the same as before, but now it's invisible. When the help button is pressed, a message appears: **"All's in former place, but the coder forgot to paint it. He's fired."** Gameplay has same mechanics as **Stage 1** and invisible objects still work.



Figure 13: Modified Stage Design

Screenshots from Gameplay



Figure 14: Game Instance



Figure 15: Passing Stage Screen